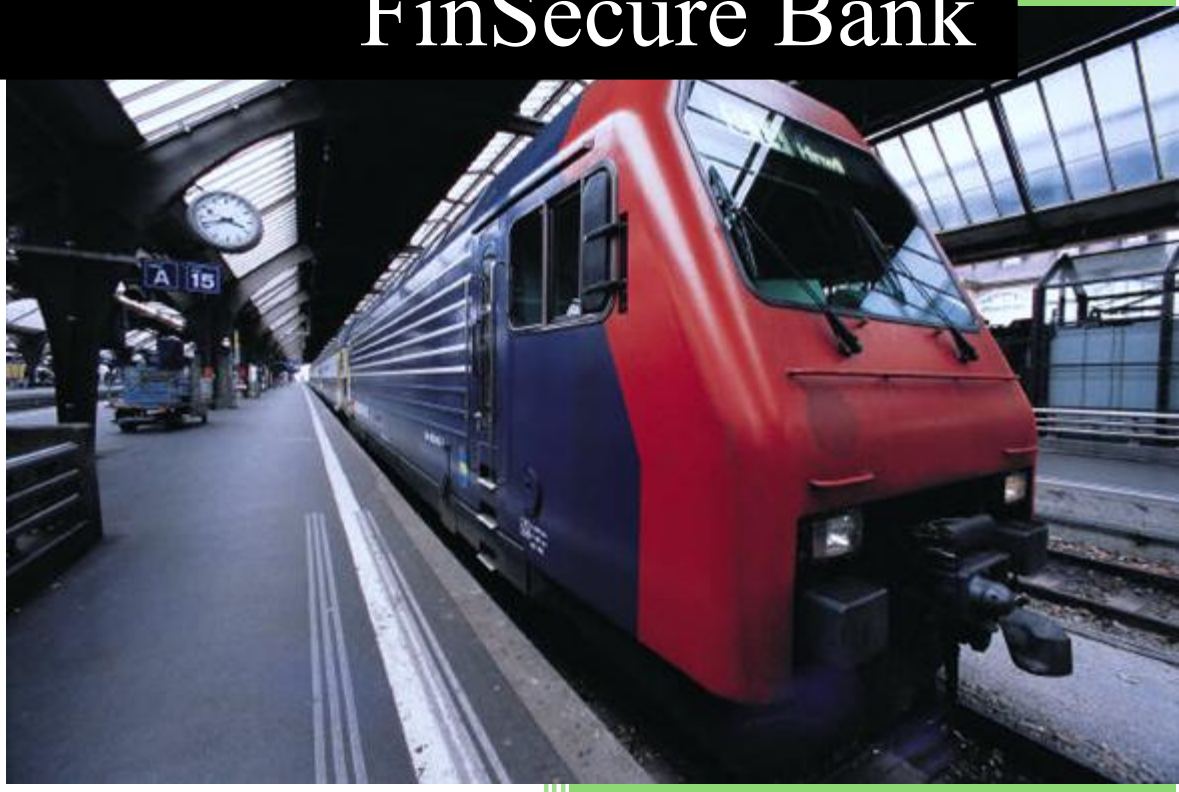
2025

Rapport d'audit de sécurité – FinSecure Bank



Eg Elisée LATHRO
Amenzo HAMICi
EFREI Paris
11/12/2025

A l'attention de

Nietch Edirisinghe	Executive Officer	+33 7 42 65 19 72	n.edirisinghe@finsecure.com

Auditeur

Eg Elisee LATHRO	Pentester Junior	+33 7 45 70 82 51	eg-elisee@efrei.net
Amenzo HAMICI	Pentester Junior	+33 7 42 31 72 81	amenzo.hamici@efrei.net

Circuit de validation interne EFREI Paris

	Prénom-Nom	Fonction	Date
Rédigé par	**Eg Elisee LATHRO**	Pentester Junior	11/12/2025
Approuvé par	**Amenzo HAMICI**	Pentester Junior	11/12/2025

Version	Date	Objet de la version
V0.1	11/12/2025	Création et rédaction du document
V0.2	11/12/2025	Rédaction terminée

Table des matières

1. Introduction 3
1.1. Contexte 3
1.2. Périmètre de l'audit 3
2. Synthèse de l'audit 3
2.1. Message pour la direction non technique 3
2.2. Tableau des vulnérabilités 4
2.2.1. SQL Injection (Authentification) 4
2.2.2. Directory Traversal 4
2.2.3. Logs Analysis – Web Attack 5
2.3. Mapping réglementaire 6
2.4. Plan de remédiation 6
2.5. Considérations éthiques 7
3. Documentation technique du test d'intrusion 7
3.1. SQL Injection – Authentification 7
3.2. Directory traversal 9
3.3. Logs analysis – Web attack 12

1. Introduction

1.1. Contexte

FinSecure Bank est une institution financière européenne spécialisée dans les services bancaires en ligne pour les particuliers et les PME. La banque gère plus de 500 000 comptes clients et traite quotidiennement des milliers de transactions par carte bancaire.

FinSecure Bank a sollicité des pentesters de EFREI Paris afin de vérifier la sécurité de leurs bases de données et leur serveur Web.

Ce document de compte rendu recense les vulnérabilités découvertes et les recommandations permettant de réduire les risques associés

1.2. Périmètre de l'audit

FinSecure Bank a mis à notre disposition un site web prototype où nous devrons retrouver seulement le mot de passe de l'administrateur.

2. Synthèse de l'audit

2.1. Message pour la direction non technique

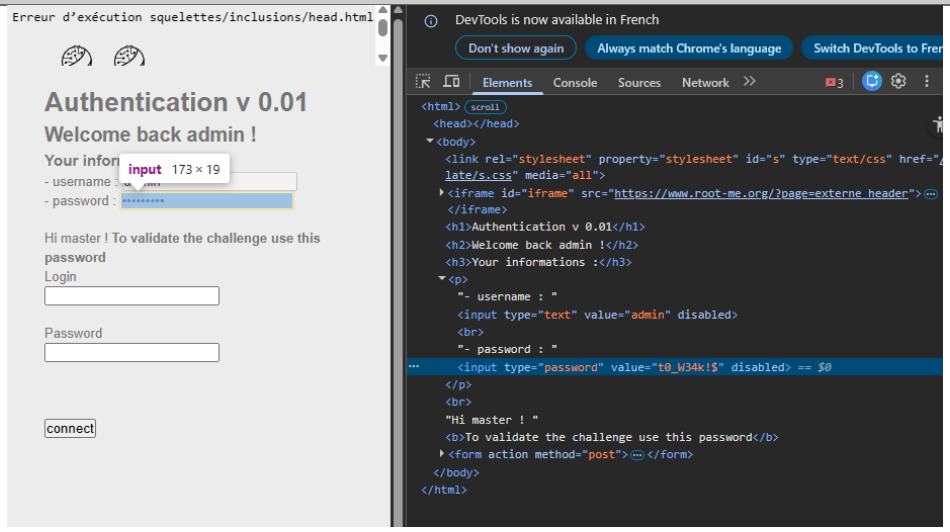
Lors de notre audit, nous avons identifié plusieurs vulnérabilités critiques sur le site prototype de FinSecure Bank :

- Une vulnérabilité d'injection SQL sur le formulaire d'authentification permettant de se connecter sans identifiants valides.
- Une vulnérabilité de type **directory traversal** exposant certains fichiers sensibles du serveur.
- Une vulnérabilité SQL dans le paramètre order de la page `/admin/?action=membres` permettant à un attaquant de récupérer le mot de passe administrateur en exécutant des sous-requêtes manuelles facile à automatiser.

Ces failles, si elles étaient présentes sur le site de production, pourraient permettre à un attaquant d'accéder à des informations sensibles. Il est donc crucial de les corriger rapidement et de renforcer les contrôles d'accès et les bonnes pratiques de sécurité.

2.2. Tableau des vulnérabilités

2.2.1. SQL Injection (Authentification)

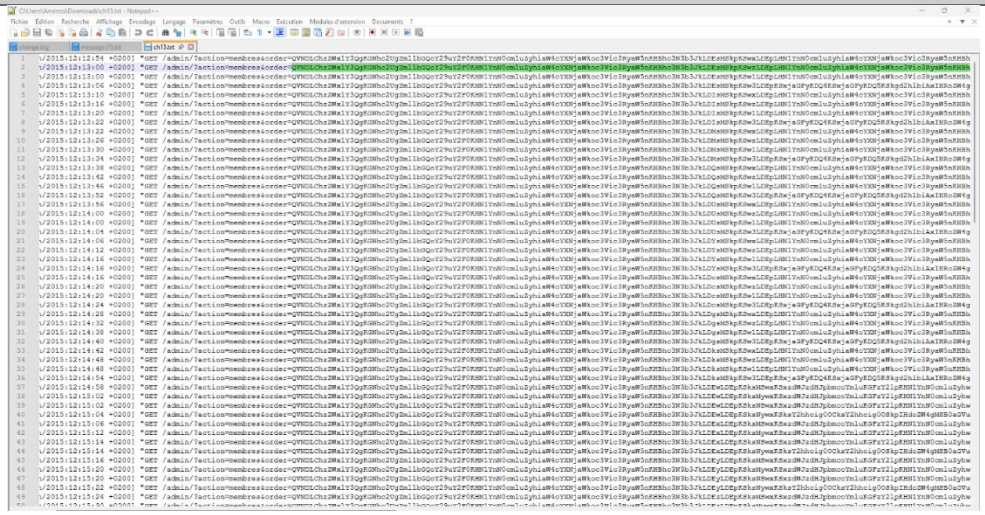
Nom	SQL Injection – Bypass d'authentification
Localisation	http://challenge01.root-me.org/web-serveur/ch9/ – Formulaire de connexion
Score CVSS 3.1	**9.8 – Critique**
Description	Le formulaire d'authentification ne filtre pas correctement les paramètres envoyés à la base SQL. L'injection `admin' --` permet de tronquer la requête et de contourner totalement la vérification du mot de passe. Cela autorise un attaquant à accéder au portail client sans identifiants valides.
Preuve	
Impact métier	Compromission totale des comptes clients ; possibilité d'usurpation d'identité ; violation de données personnelles ; exposition à des fraudes financières ; perte de confiance client pour FinSecure Bank.
Remédiation	• Implémenter des requêtes préparées (PDO, paramétrage SQL strict). • Mettre en place une validation serveur stricte des entrées utilisateurs. • Empêcher tout affichage de données sensibles en clair dans le code HTML. • Activer un WAF pour détecter/mitiger les injections.

2.2.2. Directory Traversal

Nom	Directory Traversal – Lecture non autorisée de fichiers
Localisation	http://challenge01.root-me.org/web-serveur/ch15/ch15.php – Paramètre ?galerie=
Score CVSS 3.1	**7.5 – Élevé**
Description	Le paramètre galerie est vulnérable à une attaque de parcours de répertoires (../). Cela permet d'accéder à des répertoires internes du serveur, en contournant les restrictions prévues.

Preuve	
Impact métier	Accès non autorisé à des fichiers sensibles ; risque d'exfiltration de configuration, secrets, logs ou fichiers contenant des données clients ; possibilité de pivot vers d'autres composants du SI.
Remédiation	• Interdire les caractères `../` via validation stricte côté backend. • Mapper les valeurs du paramètre sur une liste blanche de dossiers autorisés. • Désactiver l'accès direct au filesystem via un paramètre GET. • Appliquer le principe du "Least Privilege" sur le compte système exécutant l'application.

2.2.3. Logs Analysis – Web Attack

Nom	**Analyse des logs – Bruteforce automatisé et extraction de mot de passe**
Localisation	Fichier de logs encodé en Base64 fourni pour analyse
Score CVSS 3.1	**6.5 – Modéré**
Description	Les logs révèlent une attaque automatisée de type GET Flood avec des requêtes répétées : `/admin/?action=membres&orders=...` permettant de récupérer le mot de passe caractère par caractère.
Preuve	 • Mot de passe reconstruit : `g9UWD8EZgBhBpc4nTSAS`

Impact métier	Exfiltration d'un mot de passe administrateur ; risque de compromission du back-office ; perte d'intégrité des données ; possibilité de manipulation des comptes membres.
Remédiation	• Mettre en place un rate limiting sur /admin. • Activer une solution IDS/IPS ou WAF afin de détecter les patterns d'attaque. • Implémenter un mécanisme de verrouillage de compte après plusieurs tentatives. • Forcer MFA pour tous les comptes administrateurs.

2.3. Mapping réglementaire

Vulnérabilité	RGPD – Article concerné	PCI-DSS – Requirement concerné
SQL Injection – Authentification	Art. 5 (1)(f) – Intégrité et confidentialité des données Art. 25 – Privacy by design Art. 32 – Sécurité du traitement	Req. 6.5.1 – Prévention des injections Req. 3 & 8 – Protection des données & authentification
Directory Traversal	Art. 32 – Mesures de sécurité appropriées Art. 24 – Responsabilité du responsable de traitement	Req. 6.5.7 – Failures in access control Req. 7 – Restriction de l'accès aux données sensibles
Logs Analysis – Web Attack	Art. 33 – Notification des violations Art. 5(1)(f) – Sécurité des traitements	Req. 10 – Logs & monitoring Req. 6.4 – Gestion des changements & sécurisation des fichiers

2.4. Plan de remédiation

Action corrective	Priorité	Délai	Responsable
Corriger toutes les vulnérabilités SQL (authentification et paramètre order) via des requêtes paramétrées et validation stricte des entrées	P1	Immédiat	Équipe Développement / Sécurité
Bloquer le Directory Traversal et restreindre l'accès aux répertoires sensibles	P1	Court	Équipe Infrastructure / DevOps

Mettre en place un système de surveillance et d'alerte pour détecter les tentatives d'injection SQL	P2	Moyen	Équipe Sécurité / SOC

2.5. Considérations éthiques

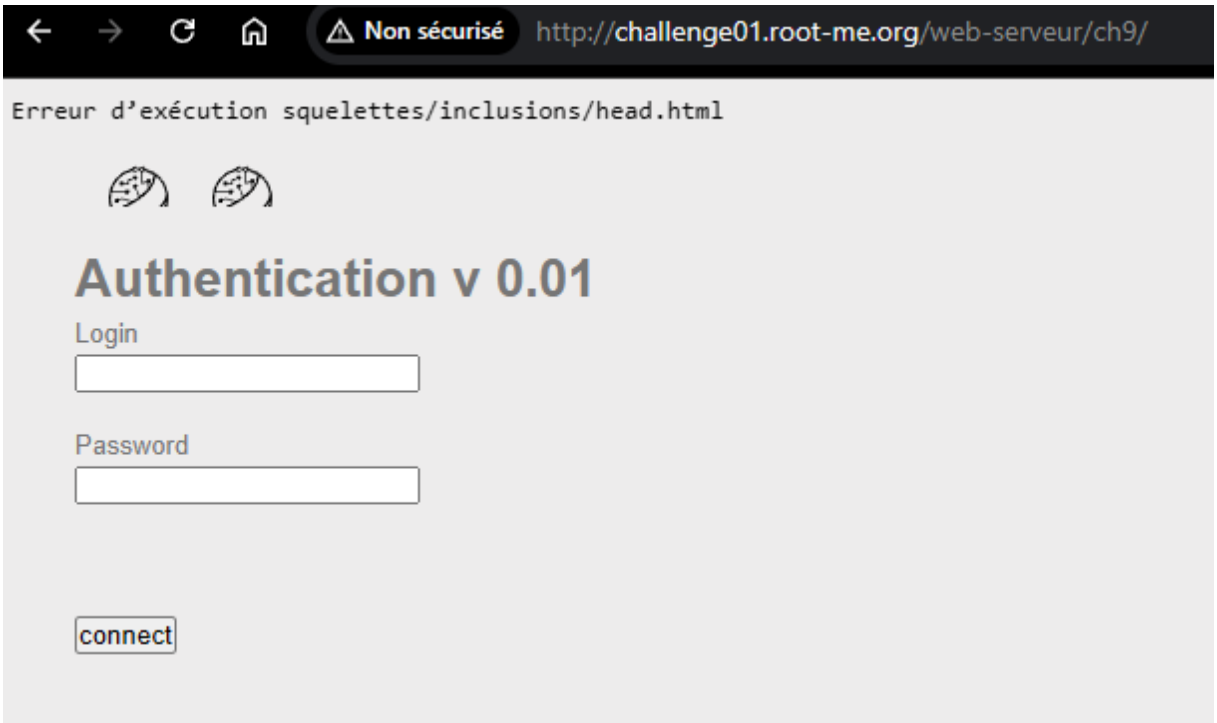
L'audit de sécurité a été réalisé dans le strict respect du périmètre défini par FinSecure Bank, à savoir le site web prototype et la recherche des mots de passe administrateurs. Toutes les données découvertes lors des tests ont été traitées de manière confidentielle et n'ont été communiquées qu'aux responsables désignés de FinSecure Bank. Aucune modification ou dommage n'a été causé aux systèmes testés : toutes les actions ont été limitées à la lecture et aux tests contrôlés. La méthodologie employée a suivi les standards reconnus de l'industrie, notamment les recommandations OWASP et le cadre PTES, afin d'assurer un audit rigoureux, reproductible et éthique.

3. Documentation technique du test d'intrusion

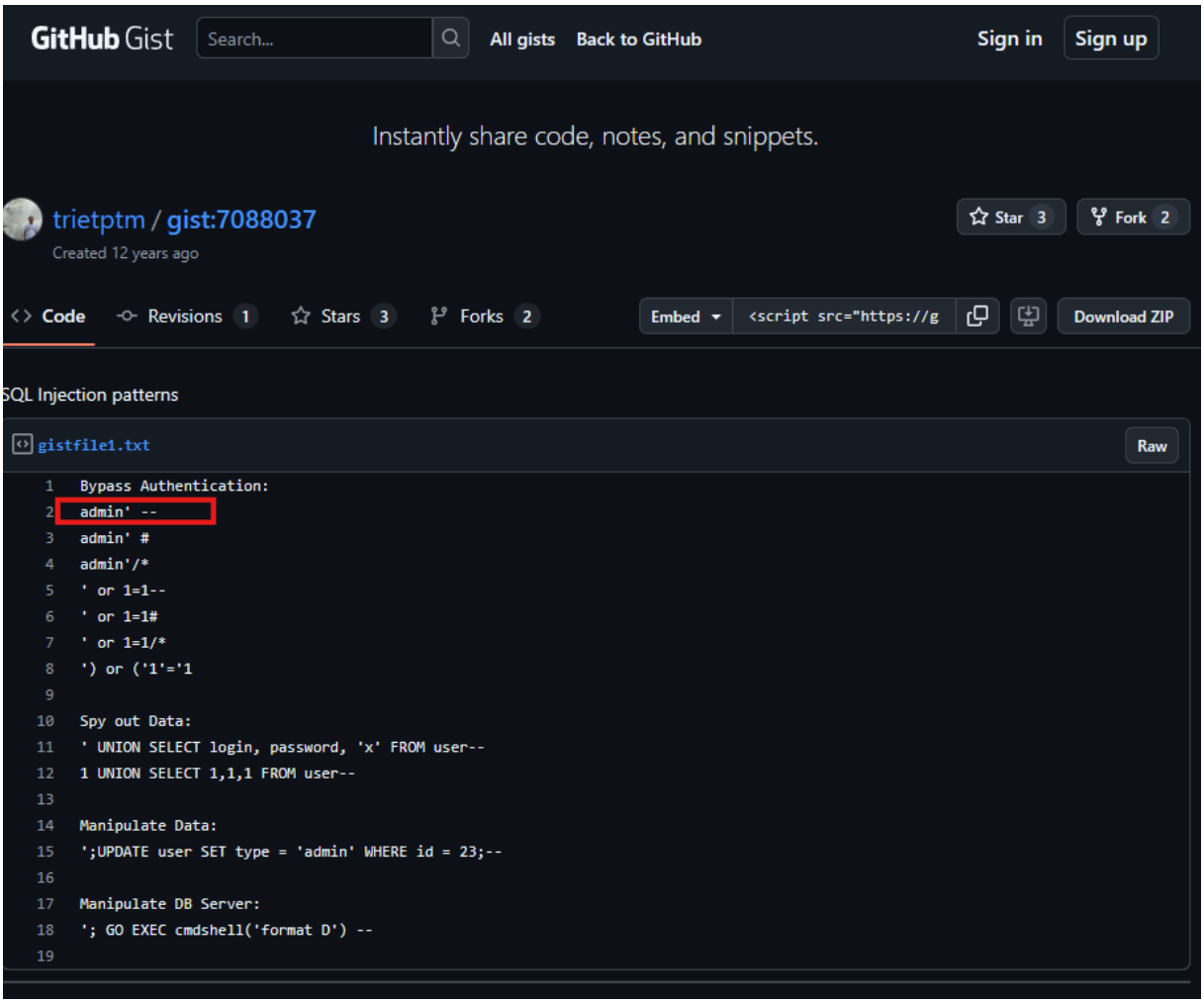
3.1. SQL Injection – Authentification

Objectif : Accéder au portail client de FinSecure Bank sans identifiants valides

Une fois le challenge lancé, on tombe sur ce site (`http://challenge01.root-me.org/web-serveur/ch9/`)

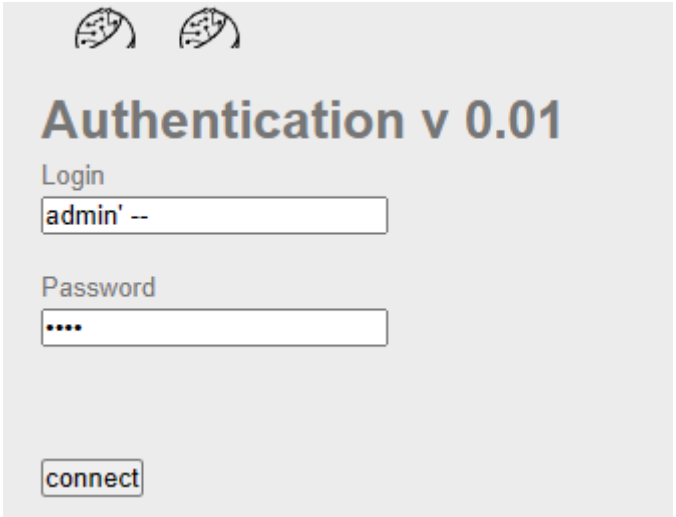


On recherche alors des injections SQL '`admin`' sur github

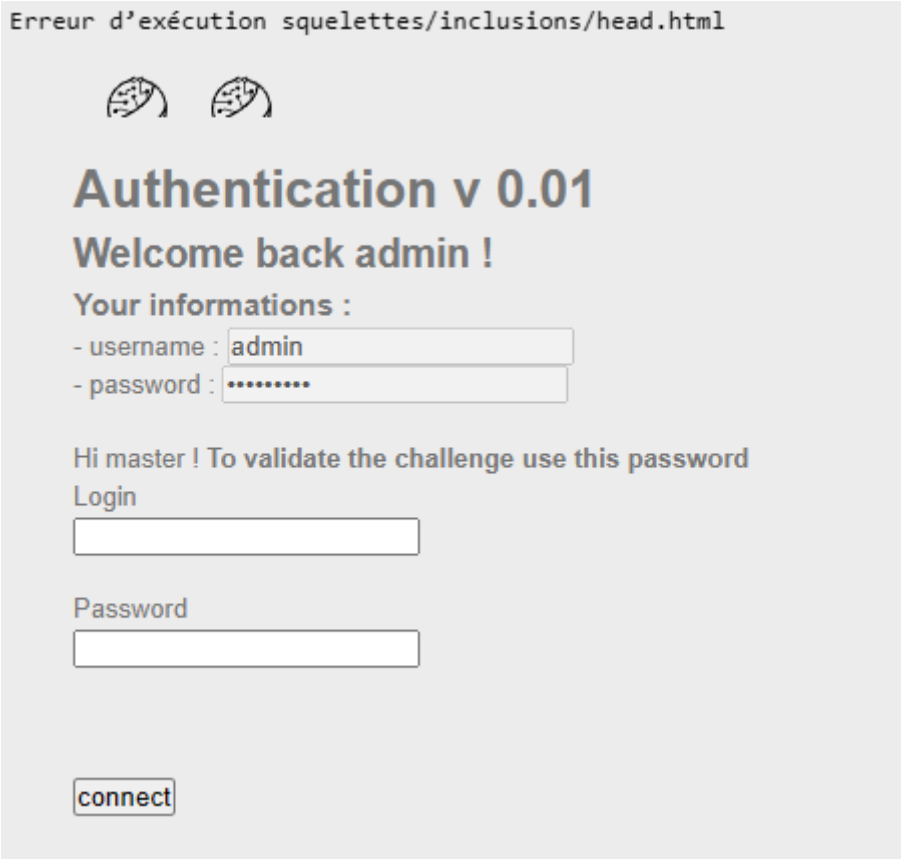


On tente alors de se connecter avec n'importe quel mot de passe :

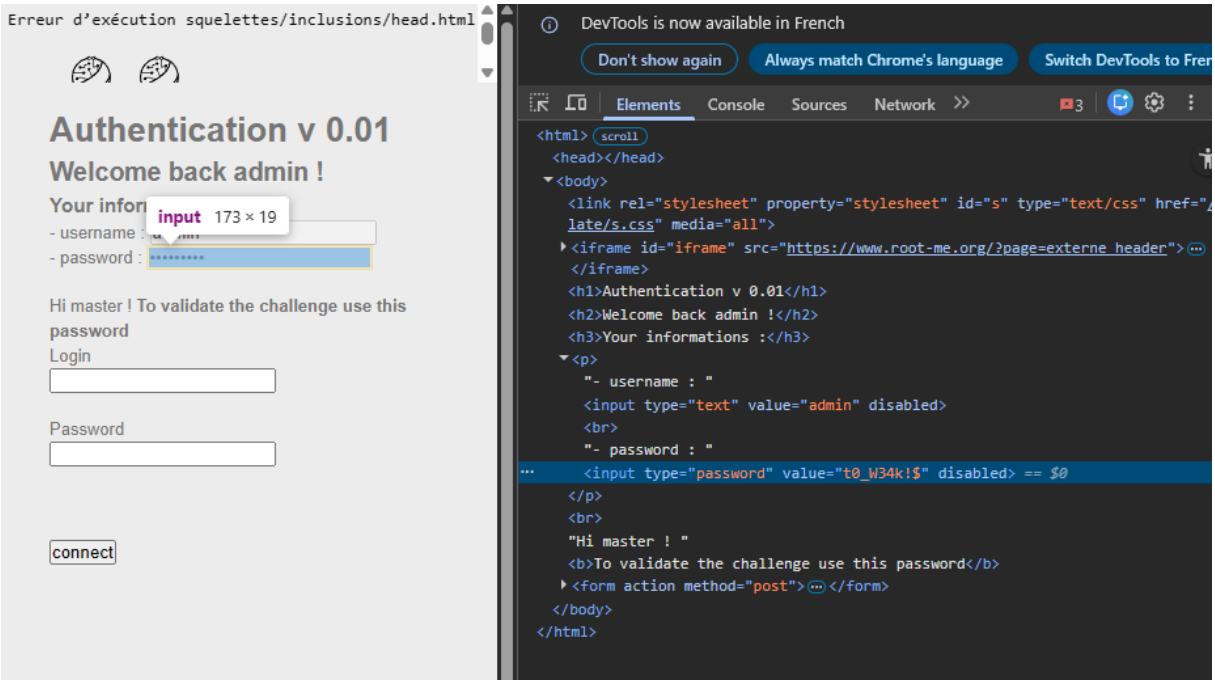
```
admin' --
test
```



L'injection SQL a marché. On tombe sur cette page :



Pour découvrir le mot de passe caché, on inspecte le code html de la page :



Le mot de passe est donc : `t0_W34k!$`

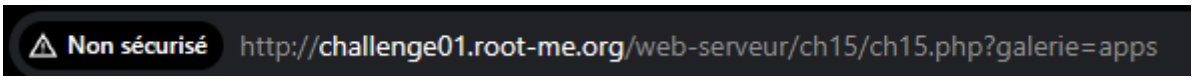
3.2. Directory traversal

Objectif : Accéder aux fichiers de configuration du serveur de FinSecure Bank

Une fois le challenge lancé, on tombe sur ce site (`http://challenge01.root-me.org/web-serveur/ch15/ch15.php`)



Nous avons décidé de cliquer sur « apps » par exemple. Et on remarque que l'url change. Les données sont enregistrées dans des répertoires qu'on peut voir dans l'URL.



Nous parcourons alors les dossiers à travers l'URL. Nous allons à la racine en écrivant :

`http://challenge01.root-me.org/web-serveur/ch15/ch15.php?galerie=../`



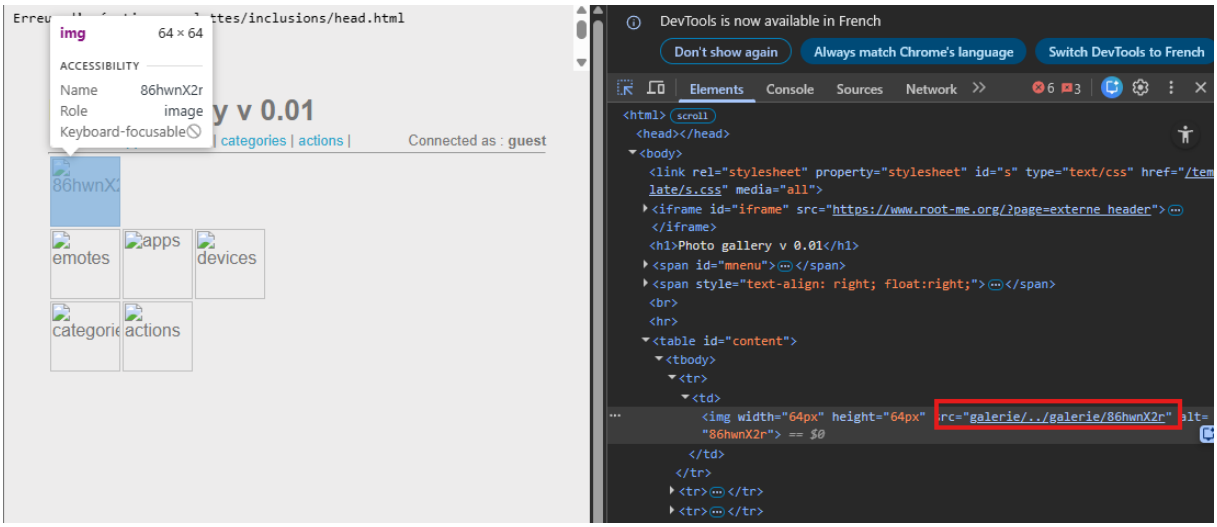
On entre dans galerie :

`http://challenge01.root-me.org/web-serveur/ch15/ch15.php?galerie=../galerie`



On tombe sur un dossier caché avec un nom ambigüe « `86hwnX2r` ».

On inspecte pour avoir le nom complet



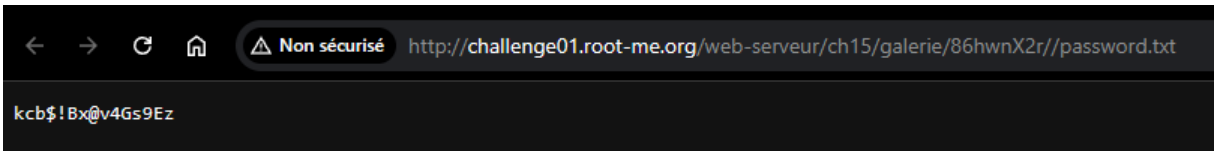
On continue de parcourir :

http://challenge01.root-me.org/web-serveur/ch15/ch15.php?galerie=../galerie/86hwnX2r/



On voit maintenant le mot de passe. On l'ouvre en faisant un clic droit dessus > "Ouvrir l'image dans un nouvel onglet"

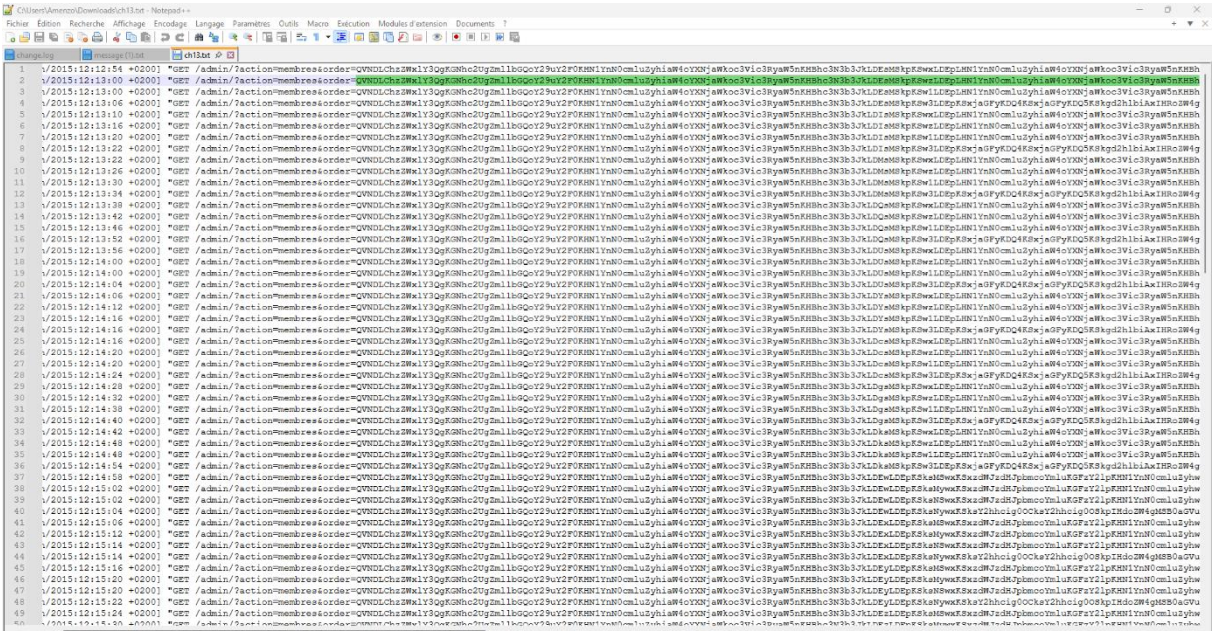
Et on trouve le mot de passe : `kcb$!Bx@v4Gs9Ez`



3.3. Logs analysis – Web attack

Objectif : Analyser les traces d'une attaque récente sur le serveur FinSecure Bank

Nous avons un fichier à notre disposition qui est encodé en base64.



En décodant les codes base64, la chaîne reconstituée est un payload d'injection SQL dans le tri des membres (`order=...`) qui sert à exfiltrer la colonne password de la table membres (ligne id = 1) en utilisant `substring`, `ascii`, `bin`, `char(48)`, `char(49)` et des `sleep()` conditionnels pour reconstruire le mot de passe.

Après décodage base64 (en tenant compte du pourcentage d'encodage URL), le paramètre order des requêtes /admin/?action=membres donne une chaîne de ce type pour le premier payload :

```
ASC,(select (case field(
    concat(
        substring(bin(ascii(substring(password,1,1))),1,1),
        substring(bin(ascii(substring(password,1,1))),2,1)
    ),
    concat(char(48),char(48)),
    concat(char(48),char(49)),
    concat(char(49),char(48)),
    concat(char(49),char(49))
)
when 1 then TRUE
when 2 then sleep(2)
when 3 then sleep(4)
when 4 then sleep(6)
end) from membres where id=1)
```

Les autres payloads décodés suivent exactement le même schéma, en changeant uniquement :

- La position du bit dans la représentation binaire (…,3,1 / …,4,1 puis …,5,1 / …,6,1, etc.).
- La position du caractère dans le mot de passe (substring(password,2,1), substring(password,3,1), etc.).

Afin de retrouver le mot de passe on a donc décidé d’appliquer la même méthode que l’attaquant en s’aidant de l'IA pour faire le décodage pour finalement retrouver le mot de passe : `g9UWD8EZgBhBpc4nTSAS`